



System design

Data Table

★ Premium



Medium



30 mins



73 done

Report an issue 🛠

Asked at these companies



Meta



Palantir



Dropbox



Stripe

Question

Design a reusable, client-rendered data table component for a large web application. The table should support a fixed header, scrollable rows and columns, declarative column definitions, and cells that can render different kinds of data such as strings, numbers, dates, and images.

Assume the table is used primarily for reading and scanning tabular data rather than spreadsheet-style editing. Unlike the [Google Sheets system design article](#), this question is about designing a reusable table component for a shared component library, not formulas, collaboration, or spreadsheet-style cell editing semantics.

Real-life examples

These libraries demonstrate common API, layout, and scaling approaches for production data tables.

- [TanStack Table](#)
- [MUI Data Grid](#)
- [AG Grid](#)
- [Ant Design Table](#)

Requirements

The base design should satisfy the following functional and performance requirements.

- The component should be generic enough to reuse across multiple product surfaces.
- The header should remain visible while rows scroll vertically.
- The table should support horizontal scrolling when there are many columns or when columns cannot shrink further.
- Columns should be configurable declaratively and cells should support custom rendering.
- The table should accept an optional outer viewport/container width while still producing sensible default column widths.
- The component should perform well with large datasets such as thousands of rows.
- Focus on the base table design first. Features such as sorting, filtering, column resizing, and fixed columns can be discussed as extensions if time permits.

Requirements exploration

Data tables show up everywhere in modern web applications: admin dashboards, analytics views, internal tools, moderation queues, billing pages, and CRMs. A good solution to this question should balance component API design with practical frontend concerns such as layout, semantics, sizing, and rendering performance.

What is the primary usage model?

This table is meant for scanning tabular data across internal tools, admin dashboards, moderation queues, billing pages, and CRM-style surfaces. The basic design should optimize for display, readability, and reuse across product surfaces rather than spreadsheet-style editing or highly interactive cell-by-cell workflows.

What kinds of data and customization should the table support?

The column contract should cover common values while leaving rich presentation under consumer control.

- Columns should be defined declaratively by the consumer.
- Each column should be able to render a consistent cell type across the whole column.
- The base component should handle common data such as strings, numbers, dates, and images.
- Consumers should also be able to supply custom renderers for richer content such as links, badges, avatars, and status pills.
- Column-level configuration should include at least the header label, accessor logic, alignment, and sizing constraints.

This is an important clarification because a reusable table component is more naturally configured per column than per individual cell. That keeps the API predictable and makes the component easier to reuse across a shared component library.

What scrolling and sizing behavior is expected?

The layout needs one predictable contract for sticky headers, overflow, and constrained widths.

- The header should remain visible while rows scroll vertically.
- The table should support horizontal scrolling when the total column width exceeds the available viewport.
- By default, columns should use sensible best-fit widths instead of requiring every width to be hard-coded up front.
- The component should also accept an optional overall width, which means the design needs a deterministic strategy for shrinking, expanding, and truncating columns when space is constrained.
- For large-data mode, assume the table renders inside a bounded-height viewport rather than growing to fit every row.

The width behavior is one of the main design challenges in this problem. Cover both the unconstrained case and the constrained-width case, rather than just leaving sizing to “whatever the browser does”.

What scale should the basic design support?

The basic design should comfortably handle thousands of rows and dozens of columns. It is reasonable to start the architecture by assuming the caller already has the row data in memory, then discuss how the rendering model should change once the dataset becomes too large to mount all rows at once, or when rows are incrementally loaded.

That means smooth scrolling and predictable layout are non-functional requirements, not nice-to-haves. The component should be designed so that large datasets do not explode the DOM tree or make scrolling janky.

What devices and accessibility expectations should guide the first version?

Desktop browsers should be the priority because dense tabular data is most commonly consumed there. The design should still be responsive enough to avoid breaking on smaller screens, but a mobile-first dense table experience is not the main target.

Accessibility should be treated as a base requirement, not an extension. Because this is a read-heavy tabular surface, the proposed design should preserve semantic table behavior and clear header-to-cell relationships.

What is explicitly out of scope for the basic design?

The core interview answer excludes features that turn the read-heavy table into a spreadsheet or application grid.

- Inline editing and spreadsheet-like keyboard navigation.
- Formulas, computed cell dependencies, and collaboration semantics.
- Frozen columns, tree rows, grouped rows, and merged-cell layouts.
- Server-driven sorting, filtering, and pagination protocols.

- Canvas rendering as the default implementation.

Common table features such as sorting, filtering, column resizing, row selection, and summary footers are worth acknowledging, but they should be treated as follow-up extensions unless the interviewer asks to prioritize them.

Exercise Beta

A team is defining the first version of a reusable data table. Which constraints form a coherent initial feature set? Select all that apply.

- ☐ 1 Require inline editing, cell-by-cell keyboard navigation, formulas, and row selection in the base component.
- ☐ 2 Accept declarative columns for common values and consumer-provided renderers for richer cells.
- ☐ 3 Optimize for scanning read-heavy tabular data rather than spreadsheet-style editing and cell-by-cell navigation.
- ☐ 4 Support thousands of rows and dozens of columns, with a bounded-height mode that can virtualize rows.
- ☐ 5 Require the table primitive itself to own server-driven sorting, filtering, and pagination.

[Reveal answer](#)[Check answer ↩](#)

Architecture / high-level design

For the scoped version of this problem, the architecture should stay mostly on the client. The consumer passes row data and declarative column definitions into a reusable `DataTable` component, and the table is responsible for sizing columns, rendering cells, managing scroll state, and switching between full rendering and a virtualized body when the dataset is large enough to justify it.

The most important architectural decision is the rendering model. For a read-heavy table, the best default is a **semantic table-based design** rather than a spreadsheet-style grid. Native table semantics already match how assistive technologies expect tabular data to be exposed, and they align with the problem statement better than a fully custom `div`-based layout.

✓ **Default to semantic `<table>` markup over a `<div>`-based grid**

For a read-heavy tabular surface, native `<table>`, `<thead>`, `<tbody>`, `<tr>`, and `<td>` elements give correct accessibility semantics and header-to-cell relationships for free. A `<div>`-based grid has to recreate all of that behavior manually with ARIA, and is only worth the cost once the component becomes interactive enough to need managed focus or spreadsheet-style keyboard navigation.

An example usage of the component in React could look like this:

```
1  const column = createColumnHelper<User>();
2
3  const userColumns = [
4    column({ key: "name", header: "Name", accessor: (row) => row.name }),
5    column({
6      key: "joinedAt",
7      header: "Joined",
8      accessor: (row) => row.joinedAt,
9      renderCell: (value) => formatDate(value),
10   }),
11   column({
12     key: "avatar",
13     header: "Avatar",
14     accessor: (row) => row.avatarUrl,
15     renderCell: (value) => <img src={value} alt="" />,
16   }),
17 ];
18
19 <DataTable
20   rows={users}
21   columns={userColumns}
22   getRowId={(row) => row.id}
23   aria-label="Users"
24   height={600}
25   width={960}
26 />;
```

This shape keeps the component reusable across product surfaces. Rows are plain data, columns define how that data should be interpreted, and custom rendering remains a per-column concern rather than an ad hoc per-cell escape hatch.

There are many other ways to define the API for a data table, and each comes with tradeoffs.

Markup approach and tradeoffs

There are two realistic markup directions:

1. **Native `<table>`**: Better default semantics, simpler header-to-cell relationships, and a more natural fit for read-heavy tabular content.
2. **Custom `div`-based layout with ARIA roles**: More control over custom layout and virtualization, but significantly more accessibility and focus behavior has to be recreated manually.

The recommended starting point is native table semantics. In the scoped version of this problem, that recommendation should stay true even when row virtualization is enabled: render the visible rows plus top and bottom spacer rows inside the same `<tbody>` so the component keeps one real table, one sticky header, and one scroll container.

A `div` or `grid`-style implementation becomes more appropriate only once the component needs absolutely positioned rows, spreadsheet-like keyboard navigation, cell editing, or aggressive two-axis virtualization. In an interview setting, those tradeoffs are follow-up material rather than the default architecture.

A simplified native structure for the basic design might look like this:

```
1 <div class="table-scroll-container" style="overflow: auto;">
2   <table>
3     <caption>
4       Users
5     </caption>
6     <thead>
7       <tr>
8         <th scope="col">Name</th>
9         <th scope="col">Joined</th>
```

```
10     <th scope="col">Status</th>
11   </tr>
12 </thead>
13 <tbody>
14   <tr>
15     <td>Ada Lovelace</td>
16     <td>Jan 3, 2024</td>
17     <td>Active</td>
18   </tr>
19 </tbody>
20 </table>
21 </div>
```

This example is intentionally simple, but it makes the structure explicit: the scrollbars belong to the outer scroll container, while the inner table preserves the semantic relationship between headers, rows, and cells.

The scroll wrapper must also be keyboard-reachable and show a visible focus indicator. Add `tabindex="0"` when the supported browsers do not otherwise include the scroller in sequential focus navigation, but omit it when it would create a redundant tab stop. Add `role="region"` and an accessible name only when the scroll area benefits from being announced as a landmark, without duplicating the table caption.

The `div` route is still worth acknowledging as a fallback path. `role="table"` fits a mostly read-only surface that needs more layout control than native tables provide, while `role="grid"` becomes appropriate only once the component is interactive enough to require managed focus and directional keyboard navigation. If the internals ever move away from native table flow, metadata such as `aria-rowcount`, `aria-colcount`, `aria-rowindex`, and `aria-colindex` may be needed to preserve accessibility. That is a later escalation point, not the recommended starting architecture.

Scroll and sticky-header strategy

The table should render inside **one bounded scroll container** that owns both vertical and horizontal overflow. The header row stays visible by using sticky positioning relative to that container, while the body rows scroll underneath it.

Architecturally, the important choice is to keep one shared scroll surface rather than splitting the header and body into separate scroll surfaces. That avoids scroll-sync bugs, duplicated width calculations, and alignment drift.

★ **Do not split the header and body into separate scroll surfaces**

A shared scroll container with a sticky `<thead>` is the simpler and more robust default. Two separate scroll surfaces require manual scroll synchronization on every scroll event, and any hiccup shows up as visibly drifting column alignment, exactly the kind of bug that is hard to debug in production and that you can avoid by letting one container own both axes.

That bounded-height detail should be explicit in the API contract. Sticky headers and row virtualization only make sense when the component has a real vertical viewport, so `height` should be treated as required for scrollable mode. If `height` is omitted, the table can still render as a plain non-virtualized table that grows naturally, but fixed-header behavior is inactive because there is no containing scroll surface for `position: sticky` to latch onto.

It is also important to distinguish the **scroll viewport width** from the **resolved table content width**. The optional `width` prop should size the outer scroll container, while the inner table may still resolve to a wider width and overflow horizontally if its columns cannot shrink any further.

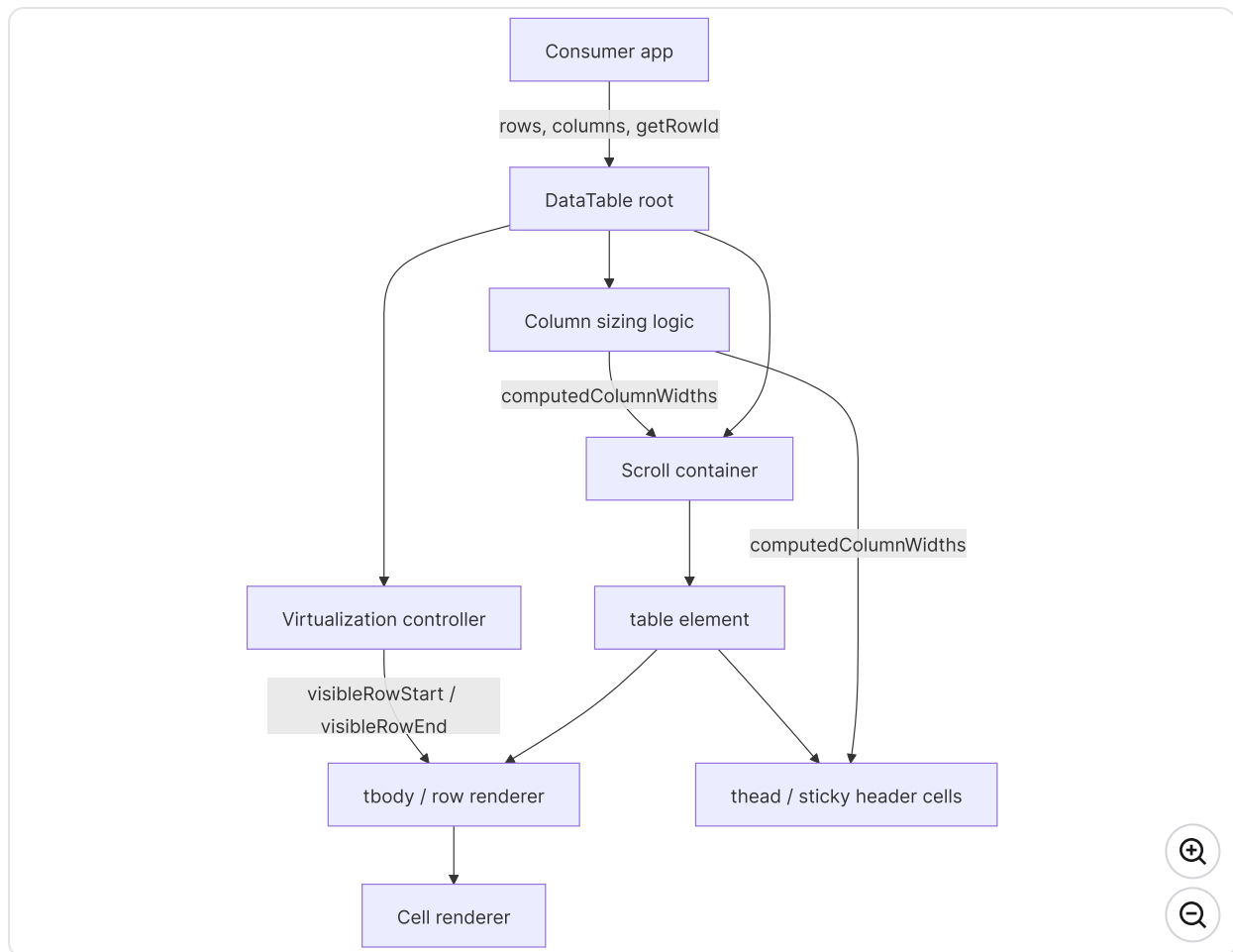
Component responsibilities

Each part of the component owns one part of rendering, sizing, scrolling, or virtualization.

Component	Role
<code>DataTable</code> root	Accepts external props, resolves static versus virtualized-body layout mode, owns derived scroll and sizing state, and coordinates the subcomponents.
Scroll container	Defines the bounded viewport and handles both horizontal and vertical scrolling.

Component	Role
Header row / header cells	Render column labels, expose header semantics, and remain visible with sticky positioning.
Row renderer	Renders the visible rows using stable row identity.
Cell renderer	Applies the default or custom renderer for each column while preserving alignment and overflow rules.
Virtualization controller	Determines which rows should be mounted based on the viewport and overscan window, and inserts spacer rows when native-table virtualization is active.
Column sizing logic	Computes default widths, applies explicit min/max constraints, handles constrained-width redistribution, and exposes one shared width map to the header and body.

The result is a relatively simple architecture: a semantic table surface, a shared column sizing model, one scroll container, and a virtualization layer that can reduce DOM cost without changing the table's external API.



DataTable component hierarchy and shared state

Exercise Beta

Which responsibility split best fits the scoped table's sizing, scrolling, and rendering work?

- 1 Use currently mounted header and row elements as the authoritative column and row model.
- 2 Let the `DataTable` root derive column widths and the visible row window, then pass them to semantic renderers.
- 3 Let each header and body cell measure its own width, while the scroll wrapper independently decides which rows to mount.



Reveal answer

Check answer ←

Data model

For a reusable data table component, the data model is a mix of **consumer-provided inputs** and **internally derived layout state**. The table should not mutate the caller's row data. Instead, it should treat the input rows and column definitions as the source of truth, then derive sizing, scrolling, and virtualization state from them.

Core external data structures / config / props

The two most important external structures are the row collection and the column definition model.

```
1  type ColumnDef<Row, Value> = {
2    key: string;
3    header: ReactNode | ((column: ColumnDef<Row, Value>) => ReactNode);
4    accessor: (row: Row) => Value;
5    renderCell?: (value: Value, row: Row) => ReactNode;
6    width?: number;
7    minWidth?: number;
8    maxWidth?: number;
9    flex?: number;
10   align?: "start" | "center" | "end";
11   overflow?: "truncate" | "clip" | "wrap";
12 };
13
14 // Heterogeneous storage type used only after each definition is checked
15 type AnyColumnDef<Row> = ColumnDef<Row, any>;
16
17 declare function createColumnHelper<Row>(): <Value>({
18   definition: ColumnDef<Row, Value>,
19 }) => ColumnDef<Row, Value>;
```

A column helper infers `Value` independently for each definition, so a date formatter receives a date and an image renderer receives a URL without casting from `unknown`. The heterogeneous collection erases that value type only after each accessor and renderer pair has been checked; consumers do not write `any` or add casts.

This shape reflects an important design decision: columns are first-class configuration objects, not something inferred ad hoc from the first row of data. That makes the component much easier to reason about across a large codebase.

Core internal state

Store measured viewport and scroll inputs; derive identities, widths, and the visible window from those inputs and the external props.

State	Type	Description
<code>scrollLeft</code>	<code>number</code>	Horizontal scroll offset of the scroll container.
<code>scrollTop</code>	<code>number</code>	Vertical scroll offset of the scroll container.
<code>viewportWidth</code>	<code>number</code>	Visible width of the scroll container.
<code>viewportHeight</code>	<code>number</code>	Visible height of the scroll container.

Presentation state comes from the external `status` prop rather than being inferred from renderer slots. The component's internal mutable state remains layout-oriented rather than data-mutating.

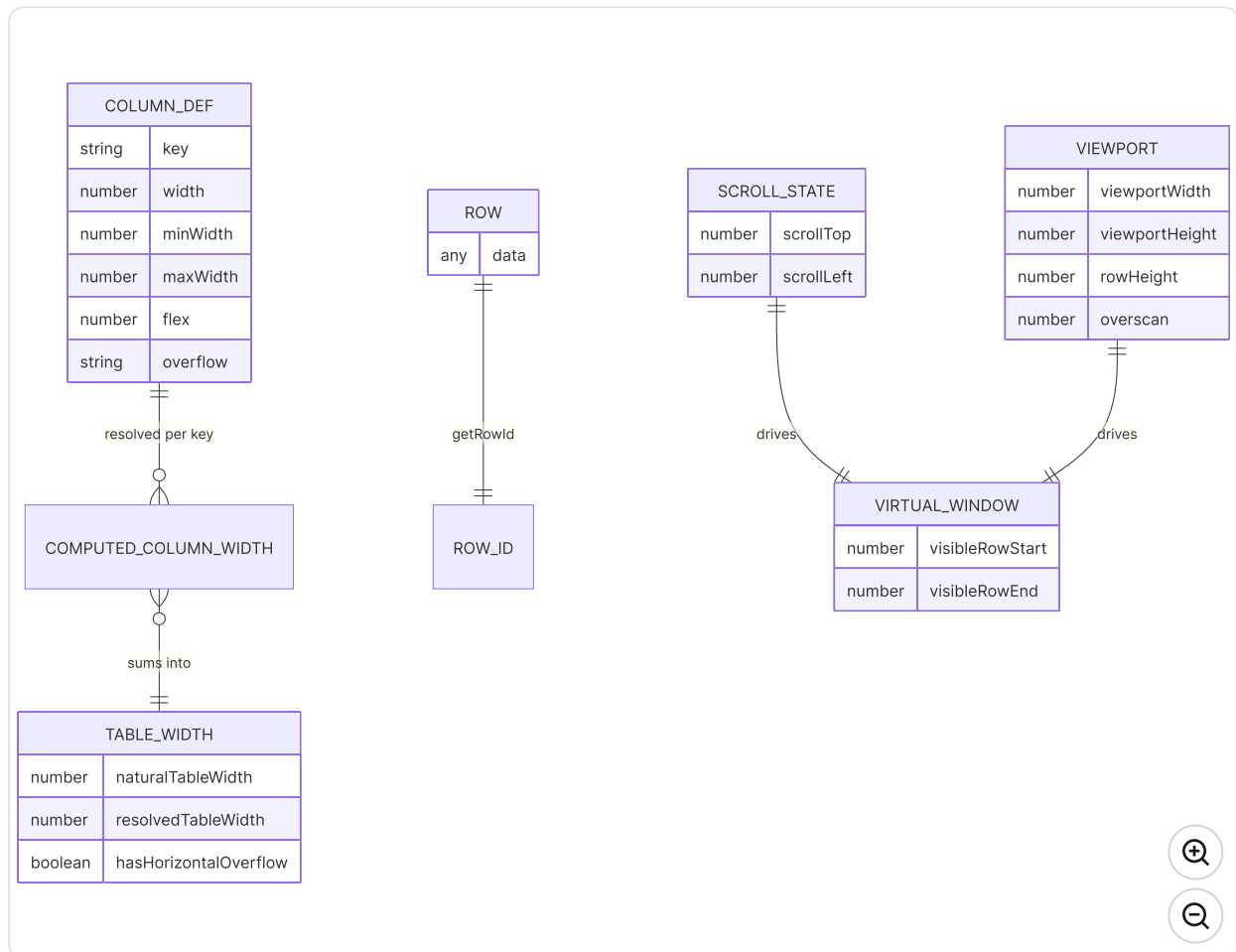
Derived state and layout metadata

Not every value needs to be stored explicitly. Several important values can be derived from the external inputs and the core state above:

- Stable `rowIds` derived from `rows` and `getRowId`
- `computedColumnWidths` derived from column constraints, sampled content, and viewport width
- `visibleRowStart` and inclusive `visibleRowEnd` derived from scroll position, viewport height, row height, and overscan
- Total logical row count
- Total scrollable body height
- Natural table width before any constrained-width redistribution
- Total resolved table width after grow/shrink logic
- Whether horizontal overflow is active

- Whether virtualization should be enabled for the current dataset size
- Resolved overflow behavior for each column after applying defaults and virtualization constraints

Treating these as derived values keeps the component easier to reason about and reduces the risk of duplicated state getting out of sync.



Relationship between consumer inputs, core state, and derived layout

Row height model

The basic design should assume a **fixed row height**. That keeps the viewport math straightforward because the visible window can be calculated from `scrollTop`, `rowHeight`, `viewportHeight`, and `overscan` without measuring every mounted row.

Variable-height rows are possible, but they push the design into a different complexity tier. Once row heights depend on rendered content, the table needs measurement logic, cached row metrics, and more complicated virtualization behavior.

For these reasons, variable row heights are better treated as an extension rather than part of the base architecture.

That assumption also affects content overflow behavior. In the fixed-row-height base design, virtualized rows should default to single-line truncation or clipping. Multi-line wrapping fits better in non-virtualized mode or in a variable-row-height extension. The base design should not claim to support `wrap` together with fixed-height virtualization unless it is also willing to measure row heights dynamically.

Exercise 1 of 3 Beta

A reusable table receives product data from its consumer and may mount only a subset of rows. Which source contracts belong in its renderer-independent model? Select all that apply.

- ☐ 1 A consumer-provided `getRowId` function for deriving each record's stable identity.
- ☐ 2 Copy incoming rows into mutable internal state and patch that copy separately when new props arrive.
- ☐ 3 The current `rows` collection supplied by the consumer as the authoritative records.
- ☐ 4 Column definitions containing accessors, sizing constraints, and optional renderers.
- ☐ 5 Use mounted cell elements as the authoritative source for row values and column definitions.



Reveal answer

Check answer ↩

Interface definition (API)

The API should make the happy path obvious: pass in rows, declare columns, provide a stable row identity, and optionally configure the viewport and sizing behavior. A good

table API should feel reusable across many product surfaces without requiring every consumer to understand the component's internal virtualization or layout machinery.

Core API

The core props describe source rows, normalized columns, viewport behavior, presentation state, and accessible naming.

Prop	Type	Description
rows	Row[]	Source data for the table.
columns	AnyColumnDef<Row>[]	Columns created through the typed column helper, with each accessor and renderer pair checked before heterogeneous storage.
getRowId	(row: Row) => string	Returns a stable identity for each row. Important for rendering, virtualization, and any row-level interactions. It serves a similar purpose to <code>key</code> s in React.
height	number string	Bounded viewport height. Required when the caller wants vertical scrolling, a sticky header, or row virtualization. If omitted, the table grows naturally and those behaviors stay off.
width	number string	Optional width of the outer scroll viewport. The resolved inner table can still be wider than this value, in which case horizontal scrolling remains enabled.

Prop	Type	Description
<code>rowHeight</code>	<code>number</code>	Fixed row height used by the basic design.
<code>overscan</code>	<code>number</code>	Number of extra rows to render above and below the viewport when virtualization is enabled.
<code>caption</code>	<code>ReactNode</code>	Optional visible table caption. Preferred when the design supports a visible accessible name.
<code>aria-label</code>	<code>string</code>	Direct accessible name when a visible caption is not appropriate.
<code>aria-labelledby</code>	<code>string</code>	ID reference to an accessible name supplied by surrounding UI.
<code>status</code>	<code>'idle' 'loading' 'ready' 'refreshing' 'error'</code>	Source state that selects loading, retained-data, empty, and error presentation. Defaults to <code>ready</code> .
<code>emptyState</code>	<code>ReactNode</code>	Content shown when there are no rows to display.
<code>errorState</code>	<code>ReactNode</code>	Content shown when the table cannot render normally because of an error or failed load.

Use one intended accessible-name mechanism: prefer a visible `caption`, otherwise use `aria-labelledby` when surrounding UI supplies the name, and fall back to `aria-label`. Do not pass competing naming props.

The `status` value selects presentation; `emptyState` and `errorState` only provide content. `loading` with no rows shows the initial loading state, `refreshing` retains

existing rows with `aria-busy="true"`, `ready` with no rows shows `emptyState`, and `error` shows `errorState` only when no rows remain. A refresh error with retained rows keeps them visible and announces the failure separately.

The base table should accept a plain list of row records. That mirrors how product code usually models server data already, so the component can drop into many surfaces without forcing consumers to reshape everything first.

The key sizing distinction is that `width` controls the viewport, not the sum of the column widths. The component still computes a separate resolved table width from the column model, then compares that content width against the viewport to decide whether to grow columns, shrink flexible ones, or keep horizontal overflow.

An example consumer-facing API could look like this:

```
1  const column = createColumnHelper<User>();
2
3  const sizedUserColumns = [
4    column({
5      key: "name",
6      header: "Name",
7      accessor: (row) => row.name,
8      minWidth: 180,
9    }),
10   column({
11     key: "joinedAt",
12     header: "Joined",
13     accessor: (row) => row.joinedAt,
14     renderCell: (value) => formatDate(value),
15     width: 140,
16     align: "end",
17   }),
18 ];
19
20 <DataTable
21   rows={users}
22   columns={sizedUserColumns}
23   getRowId={(row) => row.id}
24   aria-label="Users"
25   height={600}
26   width={960}
27   rowHeight={44}
```

```
28   overscan={8}  
29 />;
```

✓ Columns as a single array make reordering trivial

One of the greatest strengths of this design is that columns can be added or removed easily; you only need to modify the contents of the array passed into the `columns` prop and ensure that the row contains data accessed by that column. Other approaches that separate header and body cell definitions do not enjoy this benefit.

Column API details

Each column definition should expose a small but expressive set of fields:

- `key` : Stable identifier for the column.
- `header` : Header label or custom header renderer.
- `accessor` : Function that extracts the value for that column from a row.
- `renderCell` : Optional custom renderer for richer presentation.
- `width` : Preferred width when the consumer wants to opt into explicit sizing. This is especially useful for non-text columns such as avatars, thumbnails, or action cells whose ideal width is hard to infer from raw data alone.
- `minWidth` / `maxWidth` : Constraints for the sizing algorithm.
- `flex` : Relative grow/shrink weight. It defaults to `0` for an explicit-width column and `1` otherwise.
- `align` : Horizontal alignment for the cell content.
- `overflow` : Overflow policy for the column. In the fixed-row-height virtualized base design, `truncate` or `clip` should be the default. `wrap` is a better fit for non-virtualized or variable-height mode, and the simplest base policy is to disable virtualization when wrapping is required.

This keeps the API aligned with how engineers actually think about tables. Most customization happens per column, not at the level of individual cell instances.

Optional extension points

It is reasonable to leave room for common extensions without making them mandatory in the base API:

- `onRowClick`
- `sortField`
- `onSortChange`
- `renderFooter`
- `className` or slot-level styling hooks for the root, header, rows, and cells

These are intentionally optional. The base component should not force every consumer to think about sorting, row actions, or footers if all they need is a performant read-heavy table.

API design notes

Prefer a small, composable API over a huge prop surface. In particular:

- Keep the data contract centered on `rows`, `columns`, and `getRowId`.
- Keep sizing explicit enough to be predictable, but not so verbose that every column requires manual tuning.
- Make the accessibility contract explicit by supporting a visible `caption` and pass-through naming props such as `aria-label` and `aria-labelledby`.
- Keep advanced behavior as additive extension points instead of making the base API feel like a kitchen sink.

There is also a real API tradeoff worth calling out. A component library could expose a more markup-like composition API such as `Table`, `Table.Head`, `Table.Body`, `Table.Row`, and `Table.Cell`. That can feel nicely declarative, but it is usually more verbose and often pushes every consumer into building the row-mapping layer themselves. For a reusable product table, a `rows` plus `columns` contract is usually the more pragmatic default.

Designing good customization surfaces for reusable UI primitives follows the same general principles discussed in the [Front End Interview Guidebook's UI Components API Design Principles Section](#).

TanStack Table is a useful example of a headless table library API.

Exercise 1 of 4 Beta

Which public API best keeps a data table extensible without exposing its internals?

- 1 Accept rows, declarative column definitions, `getRowId`, optional viewport sizing, presentation status, and one accessible naming mechanism.
- 2 Register column definitions globally by table name so multiple instances can share configuration without receiving it as input.
- 3 Expose element refs for every header and cell and require consumers to implement layout by mutating those nodes.



Reveal answer

Check answer ←

Optimizations and deep dive

This section discusses production extensions that commonly follow the base table design, especially when datasets become large and cells become more customized:

- **Width allocation and layout**: This section explains column sizing modes, overflow behavior, and layout constraints for predictable table rendering.
- **Sticky header implementation details**: This section covers sticky positioning, scroll containers, z-index layering, and header behavior during scroll.
- **Row virtualization**: This section discusses rendering only the visible row window to keep large datasets responsive.
- **Column virtualization**: This section explains when horizontal virtualization is useful and why it is usually a follow-up optimization.
- **Accessibility**: This section covers semantic table structure, keyboard support, screen-reader behavior, and accessibility implications of virtualization.
- **Rich cell content**: This section discusses renderer APIs, mixed content types, truncation, alignment, and default layout rules for complex cells.

- **Further extensions:** This section lists practical table features that teams often add but are usually beyond the core interview scope.

Width allocation and layout

Column sizing is one of the hardest parts of this problem because the component has to support two different modes:

- **Natural width mode** when the table width is unconstrained.
- **Constrained width mode** when the scroll viewport is bounded and the column model may need to redistribute space.

To keep the behavior predictable, the component should track two separate values:

- `viewportWidth` : the visible width of the scroll container, derived from layout or the optional `width` prop.
- `naturalTableWidth` / `resolvedTableWidth` : the width implied by the column model before and after redistribution.

A practical sizing algorithm is:

1. Start with any explicit `width` values supplied by the consumer.
2. For columns without explicit widths, estimate a preferred width from the header text and a representative sample of cell contents.
3. Clamp every preferred width by `minWidth` and `maxWidth`.
4. Sum the preferred widths to get the natural table width.

In practice, step 2 should avoid scanning the full dataset. A better policy is to combine header width with a small sample of representative rows, or to measure cell width in a hidden sizing layer (e.g., an offscreen DOM element). Measuring every rendered value would be too costly, and constantly recalculating and adjusting widths during scroll is definitely undesirable.

Custom renderers need an extra rule. For text-like content, sampling the accessor output is often good enough. For richer cells such as avatars, badges, links with icons, or action menus, the raw value is not a reliable proxy for rendered width. In those cases, the component should either calculate an estimated width during the initial

sizing pass or expect the consumer to provide `width` or `minWidth`. That keeps rich cells predictable without requiring the table to inspect every mounted instance.

✓ **Consider Pretext for cheap multiline text measurement**

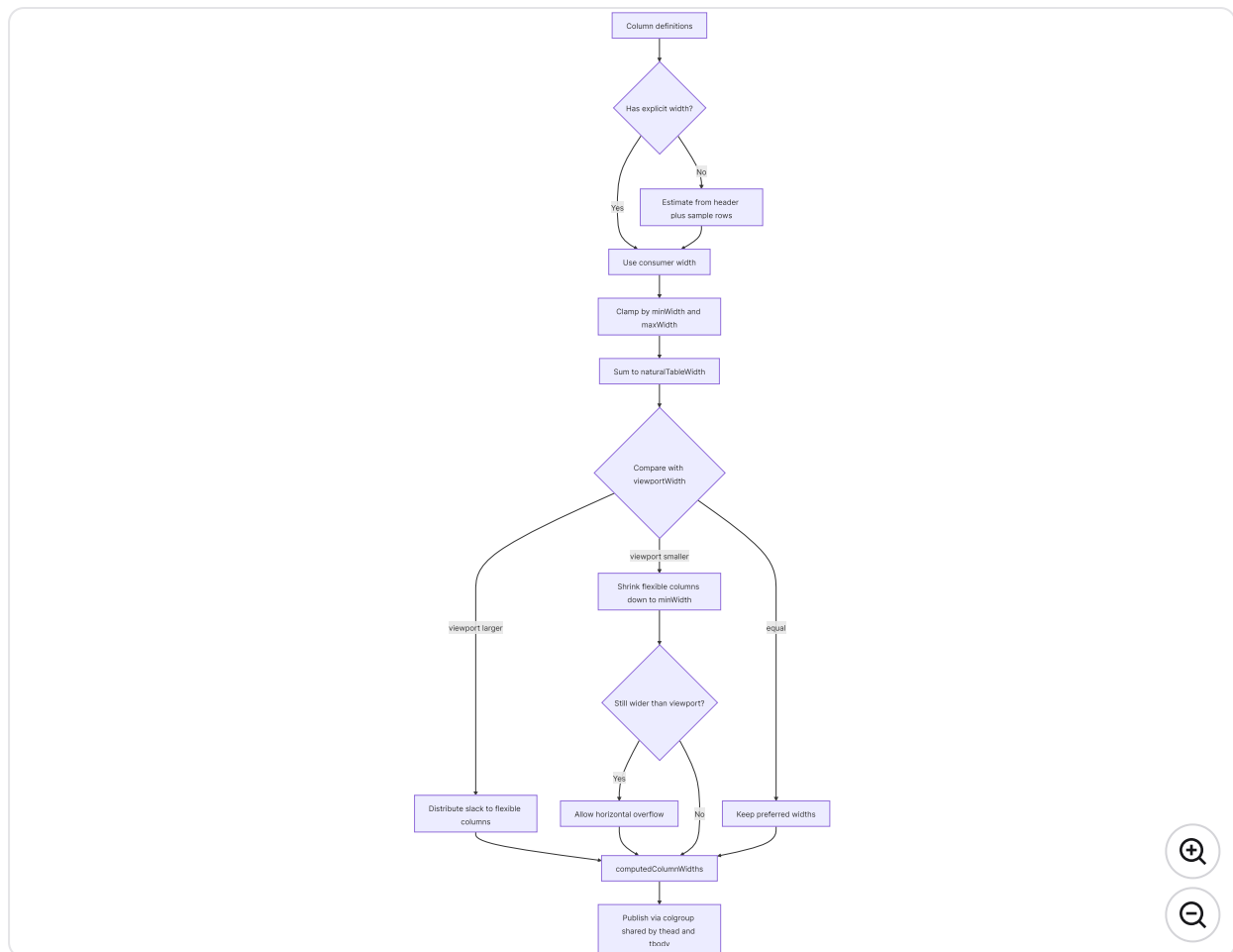
Pretext is a modern library for multiline text measurement and layout. It sidesteps the need for DOM measurements by implementing its own text measurement logic, using the browser's own font engine as ground truth. It can be used to cheaply calculate width for contents.

Once the natural width is known, compare it against the viewport width. Treat `width` as the preferred basis; only columns with `flex > 0` participate in redistribution:

- If the viewport is larger, distribute the extra space across flexible columns.
- If the viewport is smaller, shrink flexible columns proportionally until they hit `minWidth`.
- If the table is still wider than the viewport after all shrinkable columns reach `minWidth`, keep horizontal scrolling instead of over-compressing the content.

During either redistribution pass, freeze a column when it reaches `minWidth` or `maxWidth`, then redistribute the remaining difference among columns that are still flexible. Exact subpixel rounding and remainder assignment are implementation follow-ups.

That last point is important. A table should not pretend it can always fit by making every column unreadably narrow. It is usually better to preserve legibility and allow horizontal overflow than to crush the layout.



Column width resolution pipeline

Late-arriving rows introduce another subtle tradeoff. If the table keeps recomputing widths whenever new data appears, the columns will visibly jump around. A steadier default is to size from headers plus an initial sample, then let unusual outliers follow the column's overflow policy instead of immediately forcing a remeasure. In the base design, remeasurement can stay an internal policy that runs only when the column model changes, the viewport changes materially, or the implementation deliberately performs a new sizing pass.

Once the component supports explicit constraints and virtualization, application-managed sizing is easier to reason about than relying entirely on the browser's auto table layout. The browser is still useful for basic table behavior, but the sizing policy should live in the component's column model.

After the widths are resolved, they should be applied through one shared mechanism rather than allowing the header and body to size themselves independently. In native table mode, a `<colgroup>` is a clean way to publish the `computedColumnWidths` map so that both `<thead>` and `<tbody>` follow the same column model. Apply the resolved content width to the `<table>` itself before using `table-layout: fixed`; leaving the table width as `auto` can fall back to content-dependent layout. Browser

auto layout is still reasonable for a simpler non-virtualized table that mostly defers sizing to content, but it should not be the primary strategy once the component is doing explicit width allocation. If the implementation later switches to a `div` + ARIA table, that same width map can drive a shared `grid-template-columns` or explicit inline widths. The key idea is that header and body alignment come from one authoritative width source.

✓ **Pair the column width map with `table-layout: fixed`**

Once the component computes explicit column widths, `table-layout: fixed` prevents later cell content from participating in horizontal width resolution. That keeps sticky-header alignment, virtualized row insertion, and truncation behavior consistent as new rows mount.

Exercise 1 of 4 Beta

A table mixes fixed and weighted columns inside a resizable container. Which rules produce deterministic widths? Select all that apply.

- ☐ 1 Allocate remaining container width among flexible columns using an explicit weighting rule.
- ☐ 2 Let each cell apply its own preferred width and rely on the browser to reconcile disagreements between header and body content.
- ☐ 3 Recompute preferred widths from every newly mounted virtualized row while the user scrolls.
- ☐ 4 Clamp each column to its declared minimum and maximum width.



Reveal answer

Check answer ↩

Sticky header implementation details

The architecture already chose one bounded scroll container. At implementation time, the main concerns are sticky positioning and visual layering. Header cells can use `position: sticky` with `top: 0`, plus an explicit background color and `z-index` so scrolling rows do not bleed through visually. If the design uses borders or shadows to

separate the header from the body, those should stay attached to the sticky header cells so the boundary remains visible during scroll.

Resize handling matters here as well. When the viewport changes, the component should run one sizing pass and publish the updated widths through the shared column model so the sticky header and scrolling body remain aligned. This is another reason to avoid separate header and body scrollers unless the design has already outgrown the native-table architecture.

Exercise Beta

Which layout most reliably keeps a table header aligned while the rows scroll?

- 1 Place header and rows in the same horizontal sizing system and make the header sticky within the intended scroll container.
- 2 Keep the header outside the scroll container and update its transform from a throttled scroll listener.
- 3 Render the header in a separate table and synchronize only its horizontal scroll position with the body.



Reveal answer

Check answer ↩

Row virtualization

For datasets with thousands of rows, row virtualization is the main optimization. The core idea is the same as in **Autocomplete**: render only the visible window plus a small overscan buffer instead of mounting the whole dataset.

With fixed row height, the visible window can be computed cheaply:

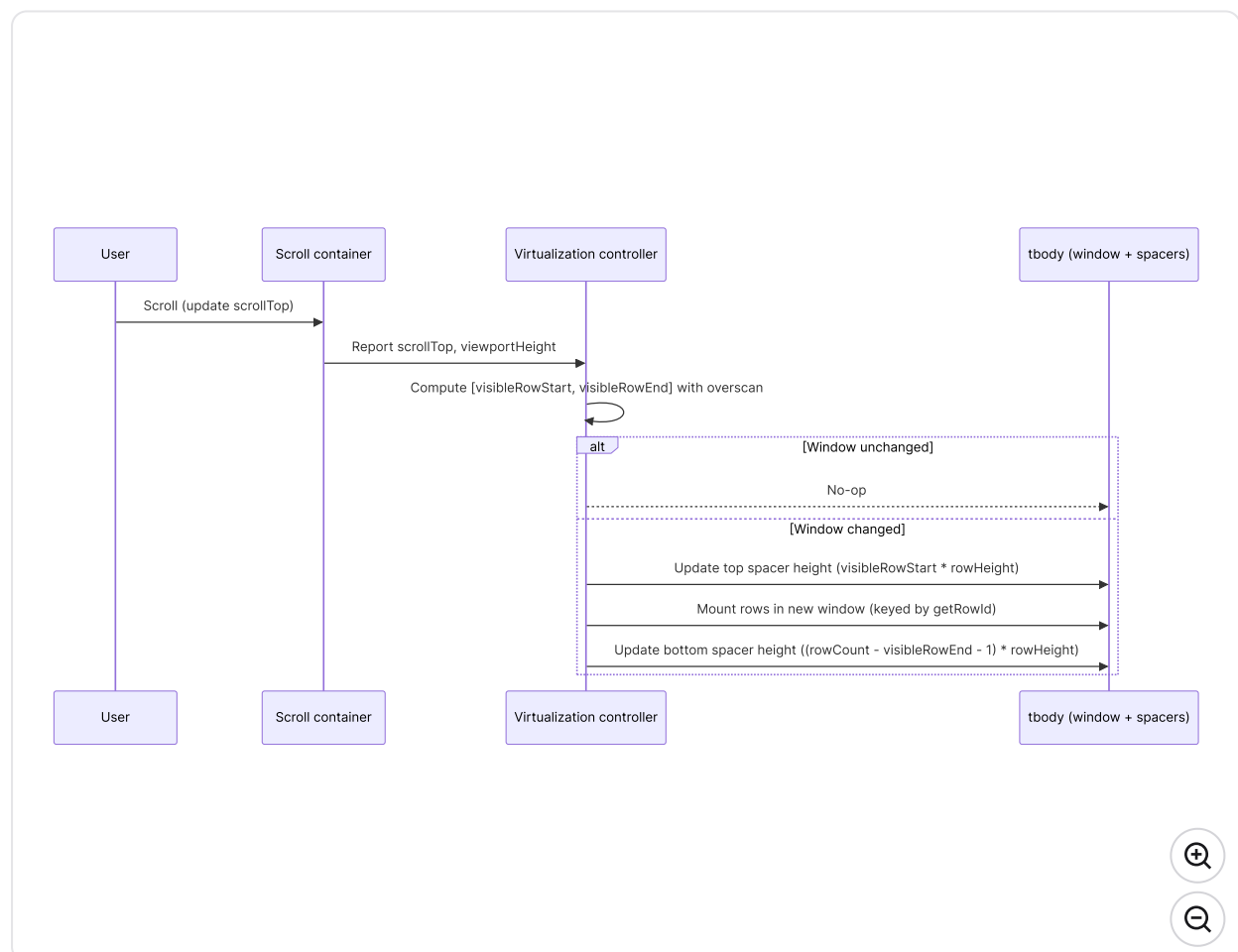
- `visibleRowStart = max(0, floor(scrollTop / rowHeight) - overscan)`
- `visibleRowEnd = min(rowCount - 1, ceil((scrollTop + viewportHeight) / rowHeight) - 1 + overscan)`

In other words, `visibleRowEnd` should be treated as an inclusive bound. The mounted window is the clamped range `[visibleRowStart, visibleRowEnd]`, not a half-open

interval.

In the scoped design, the DOM only contains that row window plus top and bottom spacer rows inside `<tbody>`, so the scrollbar still reflects the full logical height while the table keeps native header and cell relationships. Those spacer rows should be treated as implementation detail rather than real data rows: render them with one empty cell spanning all columns, keep them free of focusable content, and ensure actual data rows still carry the correct logical ordering. If a later version needs translated layers or absolutely positioned rows for more aggressive windowing, that is the point where switching the internals to a `div` + ARIA table becomes more reasonable.

Because most logical rows are absent, expose the full count with `aria-rowcount` on the table and the 1-based logical position with `aria-rowindex` on every mounted header and data row. The total count includes header rows; with one header row, that row has index `1` and the first logical data row has index `2`. Keep spacer rows out of the accessibility tree. If a rich cell contains focus, keep its row mounted until focus leaves it or move focus deliberately before eviction.



Row virtualization on scroll with spacer rows

! Spacer rows must never carry focusable content or readable text

If spacer rows contain interactive elements or readable text, assistive technologies treat them as real rows and keyboard users can tab into empty space. Keep them as a single empty cell with `colSpan={columns.length}`, no focusable descendants, and `aria-hidden="true"`, so they stay a pure layout artifact of the virtualization window.

This keeps the number of mounted row nodes proportional to viewport size instead of dataset size. It also makes the performance discussion concrete:

- Stable row identities prevent unnecessary remounts.
- Fixed row height keeps the math cheap.
- Overscan prevents flashes of empty content during fast scrolling.
- Lightweight cell renderers keep scrolling smooth.

Virtualization reduces DOM size, but it does not automatically prevent expensive rerenders. In a React implementation, scroll state and viewport calculations should stay localized to the table controller, while visible rows and cells receive stable inputs whenever possible. Stable column definitions, lightweight formatter functions, and memoized row or cell subtrees can all help keep scroll work proportional to the visible window rather than redoing unnecessary work across every mounted cell.

★ Do not key rows by array index in a virtualized table

Use the stable identity returned by `getRowId` as the React key. Index-based keys cause rows to get reconciled to a different underlying record as the window shifts during scroll, which breaks row-local state, defeats memoization, and can produce subtle visual glitches where cell content lags behind the intended row.

In interviews, it is usually fine to assume the data has already been fetched and is handed directly to the component. If a real product later needs side-loading, server pagination, or infinite loading, the same `rows` and `columns` contract can remain intact while fetching stays outside the table itself. That keeps the primitive reusable instead of binding it to one transport pattern.

Exercise 1 of 4 Beta

Rows enter and leave a recycled viewport as the user scrolls through thousands of records. Which practices preserve logical row identity and scroll geometry? Select all

that apply.

1

Represent the full list height with spacers or transforms while mounting only the active window.

☐

2

Key mounted rows by stable row ID rather than the visible index.

☐

3

Render a small overscan region around the viewport to reduce blank flashes.

☐

4

Treat each recycled viewport slot as row identity and reuse that key for whichever record enters it.

☐

5

Keep measured off-screen rows mounted but hidden so variable heights never need to be estimated again.

☐[Reveal answer](#)[Check answer ↩](#)

Column virtualization

Column virtualization is a valid follow-up optimization, but it should not be the base recommendation. In most product tables, row count becomes a problem much earlier than column count.

Column virtualization also adds cost:

- Header and body alignment get trickier.
- Keyboard navigation and accessibility metadata become harder to maintain.
- Sticky regions and future fixed-column support become more complex.

If the interviewer pushes on extremely wide surfaces with hundreds of columns, column virtualization becomes more compelling. That is closer to the tradeoff space in the [Google Sheets system design article](#), where both axes are large enough to justify heavier viewport-related computation logic.

Accessibility

The basic design should keep semantic table behavior:

- Header cells should use proper table header semantics such as `<th scope="col">`.
- The table should expose one clear accessible name through a visible `caption`, `aria-labelledby`, or `aria-label`.
- When `status` is `loading` or `refreshing`, the table region or scroll container should expose `aria-busy="true"` so assistive technologies know the content is updating.
- Empty and error states should be announced in a way that makes sense for assistive technology users. If they replace the body, rendering them as a full-width row with `colSpan={columns.length}` preserves table context; if they are transient status messages, an adjacent live region may be more appropriate.

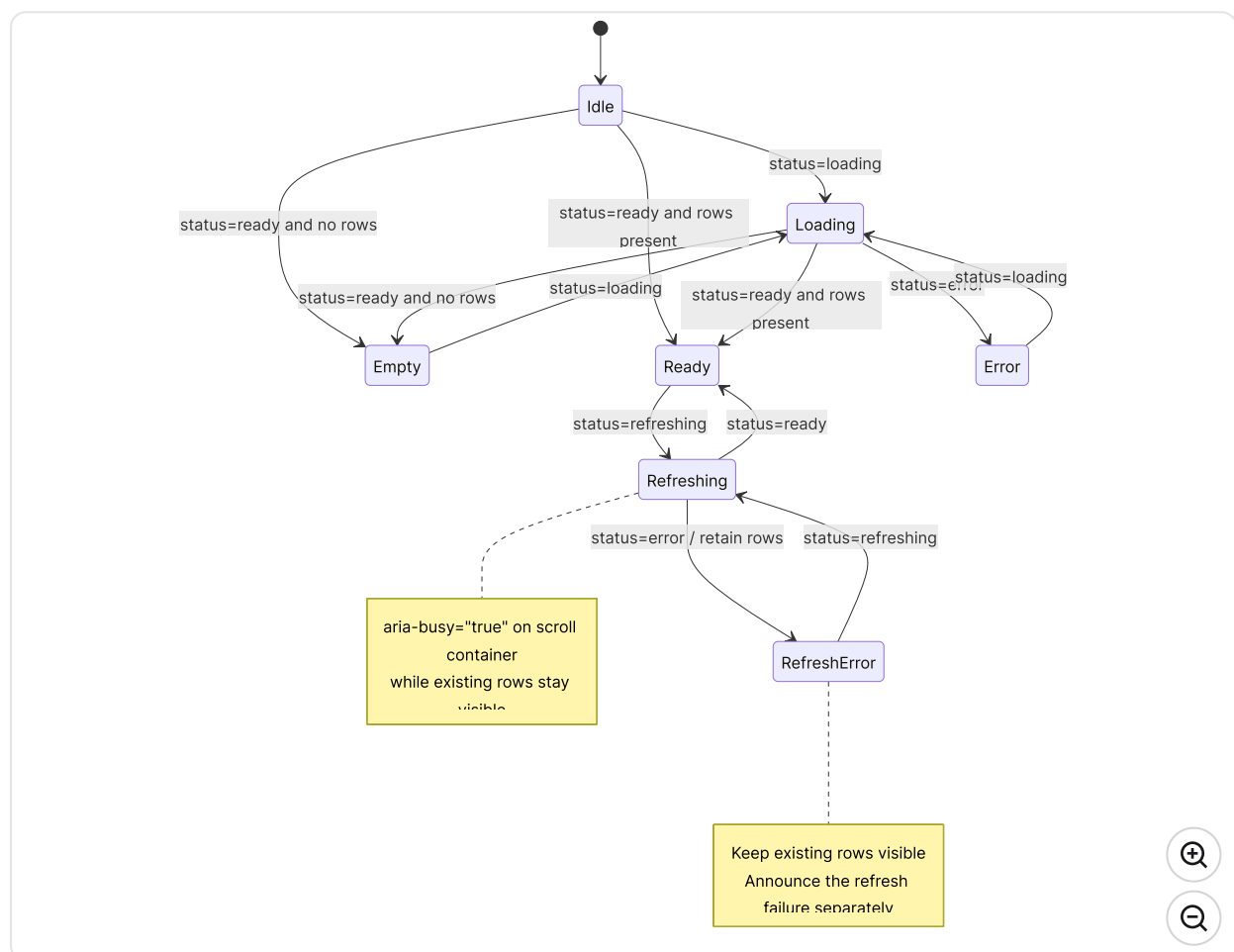


Table presentation lifecycle

Virtualization adds one important nuance: the full dataset may no longer be present in the DOM. Native table markup still needs `aria-rowcount` and logical `aria-rowindex` values when rows are omitted; spacer rows remain hidden layout artifacts. If browser and screen-reader testing shows that this implementation creates confusing row

navigation, offer a non-virtualized or paginated fallback rather than assuming that `div` markup alone fixes it.

The article should also be explicit that `role="grid"` is not the default. It becomes appropriate only when the table is meaningfully interactive and needs composite-widget keyboard behavior.

Exercise 1 of 2 Beta

The component displays read-heavy records without cell editing or directional cell navigation. Which semantic model best fits it?

- 1 Use a native `<table>` with semantic header and data cells plus one clear accessible name.
- 2 Render rows as a list of generic elements and rely on spacing and an `aria-label` to communicate columns.
- 3 Use `role="grid"`, roving cell focus, and arrow-key navigation even though cells are not interactive.

[Reveal answer](#)[Check answer ↩](#)

Rich cell content

Column renderers add support for mixed content types, but a few default layout rules should be established:

- Text-heavy columns should use an explicit overflow policy instead of overflowing unpredictably. In the fixed-row-height virtualized base design, truncation or clipping is the safer default, while multi-line wrapping fits better in non-virtualized or variable-height mode.
- Numeric and date columns often read better when aligned consistently.
- Image cells should use constrained thumbnails or avatars so they do not blow up row height.
- Image alternative text depends on context: use `alt=""` when adjacent text already conveys the same information, and require purpose-specific text when the

image adds information or performs an action.

- Rich non-text cells usually need either offscreen DOM measurements or explicit width constraints so the sizing algorithm does not guess from raw values.
- Custom renderers should be lightweight and predictable because they sit in the hot rendering path during scrolling.

Exercise 1 of 2 Beta

How should a reusable table support interactive or rich cell content?

- 1 Let each cell renderer mutate its source row while formatting the displayed value.
- 2 Accept HTML strings from column definitions so consumers can render any cell without a component API.
- 3 Allow typed render functions or components that receive the cell value and row without owning canonical row state.



Reveal answer

Check answer ↩

Further extensions

Here are other practical follow-up features that teams usually want but are typically out of scope for interviews:

- Sorting
- Filtering
- Paginated data
- Column resizing
- Sticky/frozen columns
- Sticky footers
- Row actions or row selection

Those features are compatible with the overall architecture, but they should layer on top of the same core design: declarative columns, one shared sizing model, one scroll container, and virtualization that is primarily row-oriented.

Variable-height measurement, two-axis virtualization, and the supported browser/screen-reader test matrix are useful potential follow-ups, not requirements for the core interview design.

Summary

Scope `DataTable` as a reusable, read-heavy component. It turns `rows`, declarative `columns`, and a stable `getRowId` into a semantic, accessible table capable of handling thousands of rows and dozens of columns. Sorting, filtering, pagination, selection, and inline editing stay outside the core contract and attach as additive extensions, keeping the default surface small and the performance envelope predictable.

Keep semantics and column configuration in the core contract. Native `<table>`, `<thead>`, and `<tbody>` markup lives inside a single bounded scroll container so the browser handles semantics and alignment for free, while the typed column helper keeps each `accessor` and `renderCell` pair aligned with its value type.

Share one width contract between header and body. The column model feeds a `computedColumnWidths` pipeline that resolves explicit widths, estimates the rest from headers plus a small sample, clamps against min and max bounds, and distributes remaining space. Publishing the result through a `<colgroup>` paired with `table-layout: fixed` keeps header and body widths from drifting.

Use row virtualization as the main scale lever. A fixed `rowHeight` and `overscan` derive an inclusive `[visibleRowStart, visibleRowEnd]` window, with spacer rows preserving scroll height and `getRowId` keying real rows so React reconciliation stays correct.

Let native table semantics carry accessibility until the component becomes interactive. `<th scope="col">` on headers, `aria-busy` during loading, full-width `colSpan` rows for empty and error states, and `role="grid"` held in reserve for the

interactive escalation path keep the baseline accessible without overcomplicating the default component.

Start with the semantic table structure plus a shared width contract, then present row virtualization as the single performance lever that makes the scale target realistic.

References

The references below group data-table sources by the topic they inform, so it is easier to map each source back to the part of the article it relates to: accessibility patterns, layout and CSS primitives, virtualization, production data-grid libraries, and related articles and coding questions.

Accessibility patterns

This subsection links the canonical ARIA authoring guidance for read-heavy tables versus interactive grids, which underpins the semantics choices in the article.

- WAI-ARIA APG: table pattern for the accessibility model of read-heavy tabular content.
- WAI-ARIA APG: grid pattern for the tradeoffs when a table evolves into a more interactive composite widget.
- React Aria: `useTable` for accessible table structure and reusable component patterns.

Layout and CSS primitives

This subsection covers the CSS features that sizing, sticky headers, and column widths rely on.

- MDN: `position` for sticky positioning behavior within a scroll container.
- MDN: `table-layout` for the tradeoffs between automatic and fixed table layout.

Virtualization and production data grids

This subsection links the windowing model used by row virtualization along with practical guidance from mature production data-grid libraries.

- [web.dev: virtualize large lists with react-window](#) for the core windowing model behind row virtualization.
- [MUI X: data grid - layout](#) for the relationship between bounded container sizing and virtualization.
- [AG Grid: DOM virtualisation](#) for practical production tradeoffs around viewport rendering.
- [AG Grid: row height](#) for the implications of fixed versus variable row heights on virtualization.

Related system design articles

The articles below overlap with the data table on virtualization and interactive grid surfaces.

- [Collaborative spreadsheet \(Google Sheets\)](#) for the point where a read-heavy table evolves into a more interactive grid with two-axis virtualization, cell-level semantics, and spreadsheet-style behavior.
- [Autocomplete](#) for another example where viewport rendering and virtualization become important once the result set grows beyond what should be mounted all at once.

Related coding questions

The coding questions below build up the same column-definition and table-rendering ideas from this system design article in a hands-on format.

- [Data table](#) for the basic rendering and pagination mechanics of a tabular UI.
- [Data table II](#) for adding column sorting behavior on top of the base table.
- [Data table III](#) for practicing the same column-definition and reusable API ideas discussed in this system design article.
- [Data table IV](#) for extending the generalized table with per-column filtering.

Exercises

Exercise 1 of 22 Beta

A team is defining the first version of a reusable data table. Which constraints form a coherent initial feature set? Select all that apply.

- ☐ 1 Require the table primitive itself to own server-driven sorting, filtering, and pagination.
- ☐ 2 Accept declarative columns for common values and consumer-provided renderers for richer cells.
- ☐ 3 Support thousands of rows and dozens of columns, with a bounded-height mode that can virtualize rows.
- ☐ 4 Require inline editing, cell-by-cell keyboard navigation, formulas, and row selection in the base component.
- ☐ 5 Optimize for scanning read-heavy tabular data rather than spreadsheet-style editing and cell-by-cell navigation.



Reveal answer

Check answer ↩

Discussions

↶ ↷ | Normal T_T | B I U | 🔗 | 🗑️ <> Aa | ☰ ☷ </> ☰

Type comment here...

0/1000

Post

🚩 Report an issue



✓ Mark complete



Subscribe to our newsletter

Enter your email

Notify me

Practice

- Get started
- JavaScript functions
- User interface coding
- System design

Quiz

Guides

- Front End Interview Playbook
- Front End System Design Playbook
- React Interview Playbook
- Behavioral Interview Playbook

Study Plans

- 1 Week Plan
- 1 Month Plan
- 3 Months Plan

Company

- Pricing
- Promotions
- Roadmap
- About
- Team
- Contact us
- Become an affiliate
- Careers
- Blog
- Medium
- DEV Community



Syntax error in text
mermaid version 11.14.0